

Reviewer Pack — Minimal Geometric Entropy and DPLL Proof Path Length

Entry 5b7a124b9dca · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 04:51:05 UTC

Minimal Geometric Entropy and DPLL Proof Path Length

Entry ID: 5b7a124b9dca **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 04:51:05 UTC

1. Conjecture

Field A (mathematical branch): Geometric Complexity Theory (GCT) **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Path Length)

Statement:

For every Boolean satisfiability instance φ , the geometric entropy of its associated tautological complex, denoted as $H_{\text{GCT}}(\varphi)$, is upper-bounded by a function $f(n)$ that is linearly correlated with the DPLL proof path length of φ , such that $H_{\text{GCT}}(\varphi) = O(f(n))$ and $\text{DPLL}(\varphi) = \Theta(f(n))$.

Rationale (proposer's reasoning):

Geometric Complexity Theory (GCT) has been applied to understand the complexity of problems like matrix multiplication. Its invariants, such as geometric entropy, could potentially capture structural information about Boolean satisfiability instances that is not captured by traditional complexity measures. By connecting GCT with DPLL proof path length, we may uncover new insights into the nature of NP-complete problems.

Taxonomy category: GCT-DPLL (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 66ced3ed74008ee3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every Boolean satisfiability instance, if the geometric entropy of its associated tautological complex is less than or equal to a linearly correlated function with the DPLL proof path length by Pearson's correlation coefficient ≥ 0.8 across 30 seeds, then the conjecture is supported; otherwise, it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -geometric complexity theory AND satisfiability problem AND tautological complex-DPLL proof path length IN BOOLEAN LOGIC AND geometric entropy OF tautological complex-upper bound ON geometric entropy FOR Boolean satisfiability instances AND linear correlation WITH DPLL proof path length

Top relevant hits considered: - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/2504.04416v1] Meta-Mathematics of Computational Complexity Theory - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/1108.6261v2] The complexity of admissible rules of Łukasiewicz logic - [http://arxiv.org/abs/1512.05667v3] Proof complexity of intuitionistic implicational formulas - [http://arxiv.org/abs/1007.2816v2] Trichotomy and Dichotomy Results on the Complexity of Reasoning with Disjunctive Logic Programs - [http://arxiv.org/abs/1103.5740v2] Generating and Searching Families of FFT Algorithms - [http://arxiv.org/abs/1612.02954v1] A series of maximum entropy upper bounds of the differential entropy

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_instance(n, m):
        variables = list(range(1, n + 1))
        clauses = []
        for _ in range(m):
            clause = [random.choice(variables) * (1 if random.random() < 0.5 else -1)
                      for _ in range(random.randint(1, n))]
            clauses.append(clause)
        return variables, clauses

    def dpll(instance):
        variables, clauses = instance
        assignment = {v: None for v in variables}

        def is_satisfiable():
            for clause in clauses:
                if not any(lit in assignment and (assignment[lit] == 1) for lit in clause):
```

```

        return False
    return True

def backtrack(level=0):
    if level == len(variables):
        return is_satisfiable()

    v = variables[level]
    for value in [True, False]:
        assignment[v] = value
        if backtrack(level + 1):
            return True
        assignment[v] = None

    return False

return backtrack()

def geometric_entropy(instance):
    # Placeholder function for geometric entropy calculation
    # This is a dummy implementation and should be replaced with actual GCT computation
    variables, clauses = instance
    return len(variables) * len(clauses)

n_values = [5, 10, 15, 20, 30, 40]
metric_values = []
instances_tested = 0
n_max = 0

for n in n_values:
    for _ in range(5):
        instance = generate_instance(n, random.randint(n, 2 * n))
        path_length = dpll(instance)
        entropy = geometric_entropy(instance)

        metric_values.append((entropy, path_length))
        instances_tested += 1
        n_max = max(n_max, n)

if not metric_values:
    return {
        "metric_name": "geometric_entropy",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean_entropy = sum(e for e, _ in metric_values) / len(metric_values)
mean_path_length = sum(p for _, p in metric_values) / len(metric_values)

correlation_coefficient = 0
if mean_entropy != 0 and mean_path_length != 0:
    covariance = sum((e - mean_entropy) * (p - mean_path_length) for e, p in metric_values)
    variance_entropy = sum((e - mean_entropy) ** 2 for e, _ in metric_values)

```

```

        variance_path_length = sum((p - mean_path_length) ** 2 for _, p in metric_values)
        correlation_coefficient = covariance / (math.sqrt(variance_entropy) * math.sqrt(variance_path_length))

    conjecture_holds = correlation_coefficient >= 0.8
    counterexample = "" if conjecture_holds else "correlation_coefficient < 0.8"

    return {
        "metric_name": "geometric_entropy",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.8\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13498
2	propose	ollama_remote	glm4:latest	0	0	13002
3	preregistration	ollama_remote	glm4:latest	0	0	10080
4	novelty	ollama_remote	glm4:latest	0	0	8693
5	novelty	ollama_remote	glm4:latest	0	0	16165
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16997
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11409
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14306
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13113
10	judge	ollama_remote	glm4:latest	0	0	11236

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 128500 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5b7a124b9dca.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5b7a124b9dca.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5b7a124b9dca.tar.gz` (if generated)